커널 최적화

개요

이 문서에서는 vLLM이 GPU 성능을 극대화하기 위해 사용하는 커스텀 CUDA 커널과 Triton 커널의 구현 및 최적화 기법을 상세히 분석합니다. 메모리 대역폭 최적화, 연산 효율성 향상, 그리고 FlashAttention 통합 방식에 대해 코드 레벨에서 깊이 있게 살펴봅니다.

목차

1. CUDA 커널의 역할과 중요성
2. CUDA 커널 구현 분석
3. Triton 커널 활용
4. FlashAttention 통합
5. 메모리 최적화 기법
6. 성능 프로파일링

CUDA 커널의 역할과 중요성

GPU 가속의 핵심

CUDA 커널은 vLLM의 성능 최적화에서 가장 중요한 역할을 담당합니다. CPU에서 실행되는 일반적인 PyTorch 연산과 달리, CUDA 커널은 GPU의 수천 개 코어를 활용하여 병렬 처리를 수행합니다.

vLLM의 커널 최적화 전략

vLLM은 다음과 같은 핵심 영역에서 커스텀 CUDA 커널을 활용합니다:

- **어텐션 연산**: PagedAttention과 FlashAttention 구현
- **캐시 관리**: KV 캐시의 효율적인 메모리 관리
- **활성화 함수**: SiLU, GELU 등의 고성능 구현
- **정규화**: LayerNorm과 RMSNorm의 최적화된 구현
- **양자화**: FP8, INT8 등 저정밀도 연산 지원

CUDA 커널 구현 분석

1. 캐시 관리 커널 (cache_kernels.cu)

vLLM의 캐시 관리 시스템은 메모리 효율성을 극대화하기 위해 특별히 설계된 CUDA 커널을 사용합니다.

블록 스와핑 커널

```
void swap_blocks(torch::Tensor& src, torch::Tensor& dst,

const torch::Tensor& block_mapping) {

torch::Device src_device = src.device();

torch::Device dst_device = dst.device();

cudaMemcpyKind memcpy_type;

// 디바이스 간 메모리 복사 타입 결정

if (src_device.is_cuda() && dst_device.is_cuda()) {

TORCH_CHECK(src_device.index() == dst_device.index(),

"src and dst must be on the same GPU");

memcpy_type = cudaMemcpyDeviceToDevice;

} else if (src_device.is_cuda() && dst_device.is_cpu()) {
```

```
memcpy_type = cudaMemcpyDeviceToHost;
} else if (src_device.is_cpu() && dst_device.is_cuda()) {
memcpy_type = cudaMemcpyHostToDevice;
}
// 비동기 메모리 복사로 성능 최적화
const cudaStream_t stream = at::cuda::getCurrentCUDAStream();
for (size_t i = 0; i < num_blocks; i++) {
cudaMemcpyAsync(dst_ptr + dst_offset, src_ptr + src_offset,
block_size_in_bytes, memcpy_type, stream);
}
}
```

최적화 기법:

- **비동기 메모리 복사**: `cudaMemcpyAsync` 를 사용하여 GPU 연산과 메모리 전송을 오버랩
- **스트림 활용**: CUDA 스트림을 통한 병렬 처리
- **메모리 정렬**: 블록 크기를 메모리 정렬에 맞춰 최적화

블록 복사 커널

```
template <typename scalar_t>
__global__ void copy_blocks_kernel(int64_t* key_cache_ptrs,
int64_t* value_cache_ptrs,
const int64_t* __restrict__ block_mapping,
const int numel_per_block) {
```

```
const int layer_idx = blockIdx.x;

const int pair_idx = blockIdx.y;

scalar_t* key_cache = reinterpret_cast<scalar_t*>(key_cache_ptrs[layer_idx]);

scalar_t* value_cache = reinterpret_cast<scalar_t*>(value_cache_ptrs[layer_idx])

// 소스와 목적지 블록 오프셋 계산

const int64_t src_block_offset = src_block_number * numel_per_block;

const int64_t dst_block_offset = dst_block_number * numel_per_block;

// 스레드 단위 병렬 복사

for (int i = threadIdx.x; i < numel_per_block; i += blockDim.x) {

key_cache[dst_block_offset + i] = key_cache[src_block_offset + i];

value_cache[dst_block_offset + i] = value_cache[src_block_offset + i];

}

}
```

최적화 기법:

- **코어레스드 메모리 액세스**: `__restrict__` 키워드로 메모리 별칭 최적화
- **스레드 레벨 병렬화**: 각 스레드가 독립적으로 메모리 복사 수행
- **템플릿 특화**: 다양한 데이터 타입에 대한 컴파일 타임 최적화

2. 어텐션 커널 (paged_attention_v1.cu)

PagedAttention은 vLLM의 핵심 혁신 중 하나로, 메모리 효율적인 어텐션 계산을 위한 특별한 CUDA 커널을 사용합니다.

PagedAttention 커널 구조

```
template <typename scalar_t, typename cache_t, int HEAD_SIZE, int BLOCK_SIZE,
int NUM_THREADS, vllm::Fp8KVCacheDataType KV_DTYPE,
bool IS_BLOCK_SPARSE, int PARTITION_SIZE = 0>
__device__ void paged_attention_kernel(
float* __restrict__ exp_sums,
float* __restrict__ max_logits,
scalar_t* __restrict__ out,
const scalar_t* __restrict__ q,
const cache_t* __restrict__ k_cache,
const cache_t* __restrict__ v_cache,
const int num_kv_heads,
const float scale,
const int* __restrict__ block_tables,
const int* __restrict__ seq_lens,
const int max_num_blocks_per_seq) {
// 블록 기반 어텐션 계산 로직
// ...
}
```

최적화 기법:

- **템플릿 메타프로그래밍**: 컴파일 타임에 헤드 크기별 최적화
- **공유 메모리 활용**: 빈번히 접근하는 데이터를 공유 메모리에 캐싱
- **워프 레벨 최적화**: 32개 스레드 단위의 효율적인 연산

3. 활성화 함수 커널 (activation_kernels.cu)

활성화 함수는 트랜스포머 모델에서 빈번히 사용되므로, 고도로 최적화된 CUDA 구현이 필요합니다.

SiLU 활성화 함수

```
template <typename T>
__device__ __forceinline__ T silu_kernel(const T& x) {
// x * sigmoid(x) = x / (1 + exp(-x))
return (T)(((float)x) / (1.0f + expf((float)-x)));
}
```

GELU 활성화 함수

```
template <typename T>
__device__ __forceinline__ T gelu_kernel(const T& x) {
const float f = (float)x;
constexpr float ALPHA = M_SQRT1_2;
return (T)(f * 0.5f * (1.0f + ::erf(f * ALPHA)));
}
```

융합 활성화 및 게이팅 커널

```
template <typename scalar_t, scalar_t (*ACT_FN)(const scalar_t&), bool act_first
__global__ void act_and_mul_kernel(
scalar_t* __restrict__ out,
```

```
const scalar_t* __restrict__ input,
const int d) {
const int64_t token_idx = blockIdx.x;
for (int64_t idx = threadIdx.x; idx < d; idx += blockDim.x) {
const scalar_t x = VLLM_LDG(&input[token_idx * 2 * d + idx]);
const scalar_t y = VLLM_LDG(&input[token_idx * 2 * d + d + idx]);
out[token_idx * d + idx] = compute<scalar_t, ACT_FN, act_first>(x, y);
}
}
```

최적화 기법:

- **커널 융합**: 활성화 함수와 곱셈 연산을 하나의 커널로 결합
- **메모리 코어레싱**: `VLLM_LDG` 매크로를 통한 캐시 최적화
- **함수 인라이닝**: `__forceinline__` 으로 함수 호출 오버헤드 제거

4. 정규화 커널 (layernorm_kernels.cu)

LayerNorm과 RMSNorm은 트랜스포머의 핵심 구성 요소로, 메모리 대역폭 최적화가 중요합니다.

RMSNorm 커널

```
template <typename scalar_t>
__global__ void rms_norm_kernel(
scalar_t* __restrict__ out,
const scalar_t* __restrict__ input,
```

```
const int64_t input_stride,
const scalar_t* __restrict__ weight,
const float epsilon,
const int num_tokens,
const int hidden_size) {
__shared__ float s_variance;
float variance = 0.0f;
// 분산 계산
for (int idx = threadIdx.x; idx < hidden_size; idx += blockDim.x) {
const float x = (float)input[blockIdx.x * input_stride + idx];
variance += x * x;
}
// 블록 리덕션으로 분산 합계
using BlockReduce = cub::BlockReduce<float, 1024>;
__shared__ typename BlockReduce::TempStorage reduceStore;
variance = BlockReduce(reduceStore).Reduce(variance, cub::Sum{}, blockDim.x);
if (threadIdx.x == 0) {
s_variance = rsqrtf(variance / hidden_size + epsilon);
}
__syncthreads();
// 정규화 적용
for (int idx = threadIdx.x; idx < hidden_size; idx += blockDim.x) {
float x = (float)input[blockIdx.x * input_stride + idx];
```

```
out[blockIdx.x * hidden_size + idx] =
((scalar_t)(x * s_variance)) * weight[idx];
}
}
```

최적화 기법:

- **CUB 라이브러리 활용**: 고도로 최적화된 블록 리덕션 사용
- **공유 메모리**: 분산 값을 공유 메모리에 저장하여 재사용
- **벡터화 연산**: FP16/BF16에 대한 패킹된 연산 지원

Triton 커널 활용

Triton의 장점

Triton은 Python과 유사한 문법으로 GPU 커널을 작성할 수 있는 언어로, 다음과 같은 장점을 제공합니다:

- **개발 생산성**: CUDA C++보다 간단한 문법
- **자동 최적화**: 컴파일러가 메모리 접근 패턴을 자동 최적화
- **타일링 최적화**: 자동으로 효율적인 메모리 타일링 수행

vLLM에서의 Triton 활용

vLLM은 다음과 같은 영역에서 Triton 커널을 활용합니다:

1. 어텐션 연산 최적화

```
@triton.jit
def attention_kernel(
```

```
Q, K, V, Out,
stride_qz, stride_qh, stride_qm, stride_qk,
stride_kz, stride_kh, stride_kn, stride_kk,
stride_vz, stride_vh, stride_vk, stride_vn,
stride_oz, stride_oh, stride_om, stride_on,
Z, H, N_CTX,
BLOCK_M: tl.constexpr, BLOCK_DMODEL: tl.constexpr,
BLOCK_N: tl.constexpr,
):
# 효율적인 어텐션 계산 로직
# 자동 메모리 타일링과 최적화
pass
```

2. 양자화 커널

```
@triton.jit
def  quantize_kernel(
input_ptr, output_ptr, scale_ptr,
M, N,
BLOCK_SIZE: tl.constexpr,
):
# FP8/INT8 양자화 로직
# 벡터화된 연산으로 처리량 최적화
```

```
pass
```

커널 개발 생산성 향상

Triton을 사용하면 복잡한 CUDA 메모리 관리 없이도 고성능 커널을 개발할 수 있습니다:

- **자동 메모리 관리**: 공유 메모리와 레지스터 할당 자동화
- **타일 크기 최적화**: 컴파일 타임에 최적 타일 크기 결정
- **벡터화**: SIMD 연산 자동 생성

FlashAttention 통합

FlashAttention 알고리즘 개요

FlashAttention은 어텐션 연산의 메모리 복잡도를 $O(N^2)$에서 $O(N)$으로 줄이는 혁신적인 알고리즘입니다:

1. **타일링 기법**: 입력을 작은 블록으로 나누어 처리
2. **온라인 소프트맥스**: 점진적으로 소프트맥스 계산
3. **메모리 재사용**: 중간 결과를 HBM 대신 SRAM에 저장

vLLM의 FlashAttention 구현

vLLM은 FlashAttention을 PagedAttention과 결합하여 더욱 효율적인 메모리 사용을 달성합니다.

FlashAttention 백엔드 구조

```
class FlashAttentionBackend(AttentionBackend):
@staticmethod
def  get_impl_cls() -> type["FlashAttentionImpl"]:
```

```
return FlashAttentionImpl

@staticmethod

def  get_metadata_cls() -> type["AttentionMetadata"]:

return FlashAttentionMetadata
```

메타데이터 구조

```
@dataclass

class FlashAttentionMetadata:

# 시퀀스 길이 정보

seq_lens: torch.Tensor

seq_lens_tensor: torch.Tensor

max_query_len: int

max_prefill_seq_len: int

max_decode_seq_len: int

# 블록 테이블 정보

block_tables: torch.Tensor

slot_mapping: torch.Tensor

# 어텐션 바이어스

attn_bias: Optional[torch.Tensor] = None
```

메모리 효율적 어텐션 계산

FlashAttention의 핵심은 메모리 계층 구조를 효율적으로 활용하는 것입니다:

1. 타일 기반 처리

```
def flash_attn_varlen_func(
q, k, v,
cu_seqlens_q, cu_seqlens_k,
max_seqlen_q, max_seqlen_k,
dropout_p=0.0,
softmax_scale=None,
causal=False,
window_size=(-1, -1),
alibi_slopes=None,
deterministic=False,
return_attn_probs=False,
):
# 가변 길이 시퀀스에 대한 효율적 어텐션 계산
# 메모리 사용량을 최소화하면서 높은 처리량 달성
pass
```

2. 캐스케이드 어텐션

```
def use_cascade_attention(
num_query_heads: int,
num_kv_heads: int,
query_lens: List[int],
```

```
use_sliding_window: bool,
use_alibi: bool,
) -> bool:
# FlashDecoding 사용 여부 결정
num_queries_per_kv = num_query_heads // num_kv_heads
use_flash_decoding = (num_queries_per_kv > 1 and
not use_sliding_window and
not use_alibi and
np.all(query_lens == 1))
return not use_flash_decoding
```

메모리 최적화 기법

1. 메모리 대역폭 최적화

코어레스드 메모리 액세스

```
// 메모리 별칭 최적화
const scalar_t* __restrict__ input
scalar_t* __restrict__ output

// 캐시 힌트 활용
#define VLLM_LDG(ptr) __ldg(ptr) // 읽기 전용 캐시 사용
```

벡터화된 메모리 액세스

```
// FP16 벡터 타입 정의
template<typename scalar_t, int width>
struct _f16Vec {
scalar_t data[width];
};

// 벡터화된 로드/스토어
auto* input_v = reinterpret_cast<_f16Vec<scalar_t, 4>*>(input);
_f16Vec<scalar_t, 4> vec_data = input_v[idx];
```

2. 공유 메모리 최적화

동적 공유 메모리 할당

```
// 런타임에 공유 메모리 크기 결정
int logits_size = padded_max_seq_len * sizeof(float);
int outputs_size = (NUM_WARPS / 2) * head_size * sizeof(float);
int shared_mem_size = std::max(logits_size, outputs_size);

// 커널 실행 시 공유 메모리 크기 지정
kernel<<<grid, block, shared_mem_size, stream>>>();
```

뱅크 충돌 방지

```
// 공유 메모리 뱅크 충돌을 피하기 위한 패딩

__shared__ float shared_data[BLOCK_SIZE + 1]; // +1 패딩으로 뱅크 충돌 방지
```

3. 레지스터 사용 최적화

레지스터 압력 관리

```
// 레지스터 사용량을 줄이기 위한 변수 재사용

float temp_var;

temp_var = compute_intermediate_1(input);

output1 = finalize_1(temp_var);



temp_var = compute_intermediate_2(input); // 레지스터 재사용

output2 = finalize_2(temp_var);
```

성능 프로파일링

1. CUDA 프로파일링 도구

NSight Compute 활용

```
# 커널 성능 분석

ncu  --set  full  --target-processes  all  python  benchmark_serving.py



# 메모리 대역폭 분석
```

```
ncu  --metrics  dram__throughput.avg.pct_of_peak_sustained_elapsed  python  app.
```

NSight Systems 활용

```
# 전체 애플리케이션 프로파일링
nsys  profile  --trace=cuda,nvtx  python  benchmark_serving.py
```

2. 성능 메트릭 측정

처리량 측정

```
@dataclass
class BenchmarkMetrics:
completed: int
total_input: int
total_output: int
request_throughput: float # 요청/초
output_throughput: float # 토큰/초
total_token_throughput: float # 전체 토큰/초
# 지연시간 메트릭
mean_ttft_ms: float # 첫 토큰까지 시간
median_ttft_ms: float
mean_tpot_ms: float # 토큰당 시간
```

```
median_tpot_ms: float
```

GPU 활용률 모니터링

```
def monitor_gpu_utilization():
import pynvml
pynvml.nvmlInit()
handle = pynvml.nvmlDeviceGetHandleByIndex(0)
utilization = pynvml.nvmlDeviceGetUtilizationRates(handle)
return {
'gpu_util': utilization.gpu,
'memory_util': utilization.memory
}
```

3. 커널 최적화 효과 분석

메모리 대역폭 효율성

```
def calculate_memory_efficiency(kernel_time_ms, bytes_transferred):
"""메모리 대역폭 효율성 계산"""
theoretical_bandwidth = 900e9  # A100: 900 GB/s
achieved_bandwidth = bytes_transferred / (kernel_time_ms / 1000)
efficiency = achieved_bandwidth / theoretical_bandwidth
return efficiency
```

연산 효율성

```
def calculate_compute_efficiency(kernel_time_ms, flops):
"""연산 효율성 계산"""
theoretical_flops = 312e12  # A100: 312 TFLOPS (FP16)
achieved_flops = flops / (kernel_time_ms / 1000)
efficiency = achieved_flops / theoretical_flops
return efficiency
```

성능 향상 효과

1. 벤치마크 결과

vLLM의 커널 최적화는 다음과 같은 성능 향상을 달성합니다:

처리량 개선

- **PagedAttention**: 기존 대비 2-4배 메모리 효율성 향상
- **FlashAttention**: 긴 시퀀스에서 10-20배 속도 향상
- **커널 융합**: 활성화 함수에서 15-30% 성능 향상

메모리 사용량 감소

- **KV 캐시 최적화**: 메모리 사용량 50-80% 감소
- **동적 배치**: 메모리 단편화 90% 이상 감소

2. 실제 워크로드 성능

다양한 모델 크기에서의 성능

| 모델 크기 | 기존 시스템 (tokens/s) | vLLM (tokens/s) | 성능 향상 |

|---|---|---|---|

| 7B | 150 | 450 | 3.0x |

| 13B | 80 | 280 | 3.5x |

| 30B | 35 | 140 | 4.0x |

| 65B | 15 | 75 | 5.0x |

배치 크기별 성능 특성

```
# 배치 크기에 따른 처리량 변화
batch_sizes = [1, 4, 8, 16, 32, 64]
throughput_improvement = [2.1, 2.8, 3.2, 3.6, 3.9, 4.1] # 기존 대비 배수
```

3. 에너지 효율성

커널 최적화는 성능 향상뿐만 아니라 에너지 효율성도 크게 개선합니다:

- **GPU 활용률 향상**: 60% → 85%
- **메모리 대역폭 활용률**: 45% → 75%
- **전력 효율성**: 와트당 처리량 2.5배 향상

결론

vLLM의 커널 최적화는 다음과 같은 핵심 기법들을 통해 획기적인 성능 향상을 달성합니다:

1. **메모리 계층 구조 최적화**: 공유 메모리, 레지스터, 캐시의 효율적 활용
2. **병렬 처리 최적화**: 워프, 블록, 그리드 레벨의 세밀한 최적화
3. **메모리 대역폭 최적화**: 벡터화, 코어레싱, 캐시 힌트 활용

4. **커널 융합**: 여러 연산을 하나의 커널로 결합하여 메모리 액세스 최소화

5. **자동 튜닝**: 하드웨어 특성에 맞는 최적 파라미터 자동 선택

이러한 최적화 기법들은 단독으로도 상당한 성능 향상을 제공하지만, 함께 사용될 때 시너지 효과를 발휘하여 기존 시스템 대비 3-5배의 성능 향상을 달성합니다. 특히 대규모 언어 모델의 추론 서빙에서 이러한 최적화는 실용적인 서비스 배포를 가능하게 하는 핵심 기술입니다.